Kafka with Python & Docker

Setup, Producer, Consumer & Debugging Notes

1. What is Kafka?

Kafka is a **distributed event streaming platform**. Think of it as a durable, high-throughput message bus where:

- **Producers** publish messages (events) to **topics**.
- **Consumers** subscribe to topics and process messages.
- Messages are stored on disk and can be replayed — they aren't deleted after consumption.

Core Concepts

Concept	What it means
Topic	A named channel/category for messages (e.g. orders).
Partition	A topic is split into partitions for parallelism. Each partition is an ordered, immutable log.
Offset	A sequential ID for each message within a partition. Consumers track their position via offsets.
Broker	A Kafka server that stores data and serves clients.
Consumer Group	A set of consumers that share the work of reading a topic. Each partition is read by exactly one consumer in the group.
Replication Factor	How many copies of each partition exist across brokers (for fault tolerance).

KRaft Mode (Kafka without Zookeeper)

Older Kafka versions required **Zookeeper** for cluster metadata (broker registration, leader election, topic config). Starting with Kafka 3.x, **KRaft mode** replaces Zookeeper with a built-in Raft-based consensus protocol.

- `KAFKA_KRAFT_MODE: 'true'` — enables KRaft.
- `KAFKA_PROCESS_ROLES: broker,controller` — this single node acts as both the data broker and the metadata controller.
- `KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093` — defines the controller quorum (just one node here).
- No Zookeeper container needed.

2. Docker Compose Setup

```
version: '3.8'
services:
  kafka:
    image: confluentinc/cp-kafka:7.7.8
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_KRAFT_MODE: 'true'
      CLUSTER_ID: '1qecqwef31-eqedf-x24'
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_LOG_DIRS: /tmp/kraft-combined-logs
    volumes:
      - kafka_kraft:/var/lib/kafka/data
volumes:
  kafka_kraft:
```

Key Environment Variables Explained

KAFKA_LISTENERS vs KAFKA_ADVERTISED_LISTENERS

- `KAFKA_LISTENERS` — the addresses Kafka **binds to** inside the container. `0.0.0.0:9092` means "listen on all interfaces."
- `KAFKA_ADVERTISED_LISTENERS` — the addresses Kafka **tells clients to connect to**. Since the producer/consumer run on the host, this is `localhost:9092`.

Variable	Purpose
KAFKA_CONTROLLER_LISTENER_NAMES	Tells Kafka which listener is for internal controller communication (port 9093). Not advertised to clients.
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR	The internal __consumer_offsets topic only needs 1 replica (single broker).
CLUSTER_ID	Unique identifier for the KRaft cluster. Must be same across all nodes.
volumes: kafka_kraft	Persists Kafka data across container restarts.

Common Commands

```
docker compose up -d              # Start Kafka
docker compose logs -f kafka      # Check logs
docker compose down -v            # Stop & remove data

# Inside the container (docker exec -it kafka bash):
kafka-topics --bootstrap-server localhost:9092 \
  --create --topic orders --partitions 1 --replication-factor 1
kafka-topics --bootstrap-server localhost:9092 --list
kafka-console-consumer --bootstrap-server localhost:9092 \
  --topic orders --from-beginning
```

3. Producer (Python)

```
import json, uuid
from confluent_kafka import Producer

producer_config = {"bootstrap.servers": "localhost:9092"}
producer = Producer(producer_config)

def delivery_report(err, msg):
    if err:
        print(f"Delivery failed: {err}")
    else:
        print(f"Delivered to {msg.topic()} : "
              f"partition {msg.partition()} : "
              f"at offset {msg.offset()}")

order = {
    "order_id": str(uuid.uuid4()),
    "user": "lara",
    "item": "frozen yogurt",
    "quantity": 10
}
value = json.dumps(order).encode("utf-8")

producer.produce(topic="orders", value=value,
                 callback=delivery_report)
producer.flush()
```

How It Works

1. **Producer(config)** — creates a producer instance. `bootstrap.servers` is just the initial contact point; the producer then discovers the full cluster topology.
2. **producer.produce(...)** — does **not** send immediately. It places the message into an internal buffer. Kafka batches messages for efficiency.
3. **callback=delivery_report** — called after the broker acknowledges the message. Fires during `poll()` or `flush()`, not at `produce()` time.
4. **producer.flush()** — blocks until all buffered messages are sent. Without this, your script might exit before delivery.

Important Producer Configs

Config	Default	What it does
acks	all	Broker acks needed. 0=fire-and-forget, 1=leader, all=all replicas.

Config	Default	What it does
linger.ms	5	Wait time before sending a batch (more messages accumulate).
batch.size	16384	Max bytes per batch.
retries	2147483647	Number of retries on transient failures.
compression.type	none	Compress messages (gzip, snappy, lz4, zstd).

Message Keys

Without a key, Kafka assigns messages to a **random partition** (round-robin). With a key, all messages sharing that key go to the **same partition**, guaranteeing ordering for that key.

```
producer.produce(
    topic="orders",
    key=order["user"].encode("utf-8"),  # key = "lara"
    value=value,
    callback=delivery_report
)
```

4. Consumer (Python)

```
import json
from confluent_kafka import Consumer

consumer_config = {
    "bootstrap.servers": "localhost:9092",
    "group.id": "my-group",
    "auto.offset.reset": "earliest"
}

consumer = Consumer(consumer_config)
consumer.subscribe(["orders"])

try:
    while True:
        msg = consumer.poll(1.0)
        if msg is None:
            continue
        if msg.error():
            print("Error:", msg.error())
            continue
        value = msg.value().decode("utf-8")
        order = json.loads(value)
        print(f"Received: {order['quantity']}x "
              f"{order['item']} from {order['user']}")
except KeyboardInterrupt:
    print("Stopping consumer")
finally:
    consumer.close()
```

How It Works

1. group.id: "my-group" — consumers with the same group ID share partitions. 3 partitions + 3 consumers = 1 partition each.

2. auto.offset.reset: "earliest" — when no prior offset exists, start from the beginning. Alternative: `latest` (only new messages).

3. consumer.subscribe(["orders"]) — registers interest in the topic. Can subscribe to multiple topics.

4. consumer.poll(1.0) — fetches one message or None (1s timeout). Also triggers offset commits and rebalancing.

5. consumer.close() — cleanly leaves the group and commits final offsets. Without this, rebalancing is delayed.

Important Consumer Configs

Config	Default	What it does
enable.auto.commit	true	Automatically commits offsets periodically.
auto.commit.interval.ms	5000	How often auto-commit runs.
session.timeout.ms	45000	If broker doesn't hear from consumer, it's considered dead.
max.poll.interval.ms	300000	Max time between poll() calls before consumer is kicked.

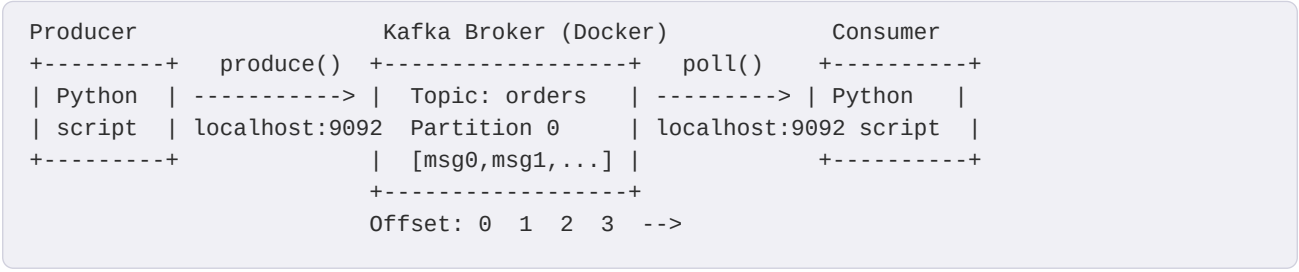
Manual Offset Commit

For more control (e.g. commit only after successful processing):

```
consumer_config["enable.auto.commit"] = False

# After processing a message:
consumer.commit(asynchronous=False)
```

5. Data Flow Summary



1. Producer serializes the order dict to JSON bytes and sends to the **orders** topic.
2. Kafka broker appends the message to partition 0, assigns it an **offset**.
3. Kafka persists the message to disk (in the Docker volume).
4. Consumer in **my-group** polls the broker, receives the message, deserializes, and prints it.
5. Consumer's offset is committed (auto-commit), so it won't re-read this message on restart.

6. Gotchas & Debugging Tips

Port already in use — If `docker compose up` fails with "address already in use" on 9092, find and kill the process: `sudo ss -tulnp | grep 9092` then `sudo kill <PID>`.

Listener typos crash KRaft — Use `KAFKA_LISTENERS` (not LISTERNERS) and colons for ports (`0.0.0.0:9092` not `0.0.0.0.9092`). KRaft throws *"controller.listener.names must contain at least one value"* if listeners config is missing.

Consumer sees nothing — Check `auto.offset.reset`. If set to `latest` and messages were produced before the consumer started, they are skipped. Use `earliest` to read from the beginning.

Messages lost on restart — Make sure you have a named volume (`kafka_kraft:/var/lib/kafka/data`). Without it, data lives only in the container's ephemeral filesystem.

Config changes not applying — Use `docker compose down -v` to remove old volumes, because KRaft stores metadata that may conflict with new configs.

f-string quote conflicts — Inside an f-string delimited with double quotes, use single quotes for method arguments: `f"...{msg.value().decode('utf-8')}"`.